Quantum Data Encoding and Feature Maps

EVA: Quantum Machine Learning – Lecture 5

Pavel Sulimov

Spring 2026

ZHAW Zürcher Hochschule für Angewandte Wissenschaften

Roadmap

1. Why encoding is the bottleneck in QML
2. Basis, amplitude, and angle encoding
3. Feature maps as unitaries and kernels as overlaps
4. Entangling feature maps, kernel estimation, and re-uploading

Bridge from Lecture 4

In Lecture 4, kernels changed geometry without explicitly listing all coordinates.

Today the same idea appears in Hilbert space:

$$x \longmapsto |\phi(x)\rangle = U_\phi(x)\,|0\rangle^{\otimes n}, \qquad K_Q(x,y) = |\langle\phi(x)|\phi(y)\rangle|^2.$$

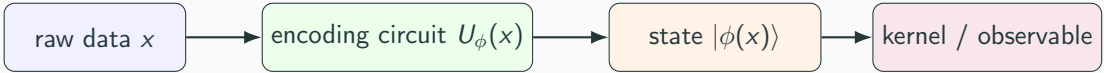
The encoding circuit decides what information is exposed to the model.

Why data encoding is a bottleneck

Classical data arrives as a vector

$$x = (x_1, \ldots, x_d) \in \mathbb{R}^d.$$

A quantum model does not act on that vector directly. It acts on a state prepared from the vector.



A poor encoding can destroy useful structure before learning even begins.

Recorded pause checkpoints

In the recorded version of this lecture, some slides are pause points for a short card set.

The five checkpoints are:

- Pause A: encoding basics
- Pause B: amplitude and angle calculations
- Pause C: overlap and fidelity
- Pause D: entangling feature maps
- Pause E: kernel estimation workflow

Data encoding strategies

Basis encoding

A binary string is mapped directly to a computational-basis state:

$$00 \mapsto |00\rangle\,, \qquad 01 \mapsto |01\rangle\,, \qquad 10 \mapsto |10\rangle\,.$$

Strengths:

- conceptually simple
- natural for discrete or already-binarized data

Limitations:

- real-valued features need preprocessing or discretization
- similarity structure is usually very rigid

`lecture_05_demo.ipynb`, Demo 1: basis, amplitude, and angle encoding states.

Amplitude encoding

A normalized vector becomes amplitudes of a quantum state:

$$x = (x_0, \ldots, x_{2^n-1}) \quad \Longrightarrow \quad |\psi(x)\rangle = \sum_{j=0}^{2^n-1} x_j \, |j\rangle \,, \qquad \sum_j |x_j|^2 = 1.$$

Strengths:

- 2^n amplitudes fit into n qubits
- very compact representation in principle

Limitations:

- normalization is mandatory
- preparing an arbitrary amplitude state can be expensive

Angle encoding

Each qubit starts in $|0\rangle$ by default, then features enter through rotation angles, for example

$$R_y(x_i) = \begin{bmatrix} \cos(x_i/2) & -\sin(x_i/2) \\ \sin(x_i/2) & \cos(x_i/2) \end{bmatrix}, \qquad R_z(x_i) = \begin{bmatrix} e^{-ix_i/2} & 0 \\ 0 & e^{ix_i/2} \end{bmatrix}.$$

Common pattern:

$$|\psi(x)\rangle = \bigotimes_{i=1}^{n} R_y(x_i)\,|0\rangle\,.$$

This is easy to implement, but the chosen rotation family determines what geometry the kernel will see.

`lecture_05_demo.ipynb`, Demo 1: basis, amplitude, and angle encoding states.

Encoding strategies at a glance

Encoding	Main idea	Strength	Main caveat
Basis	bitstring $\to \lvert z \rangle$	simple and discrete	poor for continuous data
Amplitude	values $\to$ amplitudes	compact storage	state preparation cost
Angle	values $\to$ rotations	easy circuit construction	geometry depends on gate choice

Pause A: solve Card Set A

Worked amplitude-encoding example

Suppose

$$x = (0.3, 0.7, 0.5, 0.1).$$

Its norm is

$$\|x\| = \sqrt{0.3^2 + 0.7^2 + 0.5^2 + 0.1^2} = \sqrt{0.84}.$$

Therefore the normalized state is

$$|\psi(x)\rangle = \frac{1}{\sqrt{0.84}} \left(0.3\,|00\rangle + 0.7\,|01\rangle + 0.5\,|10\rangle + 0.1\,|11\rangle\right).$$

`lecture_05_demo.ipynb`, Demo 1: explicit amplitude-encoding example.

Worked angle-encoding example

Take two features and encode them by

$$|\psi(x)\rangle = (R_y(x_1)\,|0\rangle) \otimes (R_y(x_2)\,|0\rangle)\,.$$

Since

$$R_y(\theta)\,|0\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + \sin\left(\frac{\theta}{2}\right)|1\rangle\,,$$

the two-qubit state becomes

$$\begin{aligned}|\psi(x)\rangle = {} & \cos\left(\frac{x_1}{2}\right)\cos\left(\frac{x_2}{2}\right)|00\rangle + \cos\left(\frac{x_1}{2}\right)\sin\left(\frac{x_2}{2}\right)|01\rangle \\ & + \sin\left(\frac{x_1}{2}\right)\cos\left(\frac{x_2}{2}\right)|10\rangle + \sin\left(\frac{x_1}{2}\right)\sin\left(\frac{x_2}{2}\right)|11\rangle\,.\end{aligned}$$

`lecture_05_demo.ipynb`, Demo 1: angle-encoded state expansion.

What changes when we use phase rotations?

Compare

$$R_y(\theta)\,|0\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + \sin\left(\frac{\theta}{2}\right)|1\rangle$$

with

$$R_z(\theta)\,|0\rangle = e^{-i\theta/2}\,|0\rangle\,.$$

A pure R_z on $|0\rangle$ changes only a global phase, so by itself it does not change measurement probabilities.

Phase encoding becomes useful when combined with superposition or entanglement, because relative phases can then affect overlaps and interference.

Pause B: solve Card Set B

Quantum feature maps

Feature map as a unitary

A quantum feature map is a data-dependent unitary:

$$|\phi(x)\rangle = U_\phi(x)\,|0\rangle^{\otimes n}.$$

Classical viewpoint:

$$x \mapsto \phi(x) \in \mathcal{F}.$$

Quantum viewpoint:

$$x \mapsto |\phi(x)\rangle \in \mathbb{C}^{2^n}.$$

The state itself is the lifted representation.

Quantum kernel and fidelity

For pure states $|\psi\rangle$ and $|\varphi\rangle$, the fidelity is

$$F(\psi, \varphi) = |\langle\psi|\varphi\rangle|^2 \in [0, 1].$$

Therefore a natural quantum kernel is

$$K_Q(x, y) = |\langle\phi(x)|\phi(y)\rangle|^2.$$

Interpretation:

- $K_Q = 1$: encoded states are identical up to global phase
- $K_Q = 0$: encoded states are orthogonal
- intermediate values measure partial similarity

`lecture_05_demo.ipynb`, Demo 2: exact overlaps and fidelities.

One-qubit kernel: closed-form example

Choose

$$|\phi(x)\rangle = R_y(x)\,|0\rangle = \cos\left(\frac{x}{2}\right)|0\rangle + \sin\left(\frac{x}{2}\right)|1\rangle\,.$$

Then

$$\langle\phi(x)|\phi(y)\rangle = \cos\left(\frac{x-y}{2}\right),$$

and hence

$$K_Q(x,y) = \cos^2\left(\frac{x-y}{2}\right).$$

So the overlap is large for nearby angles and smaller for separated angles.

`lecture_05_demo.ipynb`, Demo 2: one-qubit closed-form kernel.

Classical distance, classical similarity, quantum overlap

For the same two inputs we can compare three notions:

$$\|x - y\|, \qquad K_{\mathrm{RBF}}(x, y) = e^{-\gamma\|x-y\|^2}, \qquad K_Q(x, y) = |\langle\phi(x)|\phi(y)\rangle|^2.$$

They are not identical, but all three define geometry.

The design question is always the same: which similarity notion best matches the learning task?

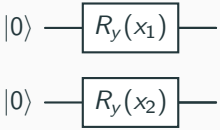
`lecture_05_demo.ipynb`, Demo 3: distance, classical similarity, and quantum fidelity.

Pause C: solve Card Set C

Entangling feature maps

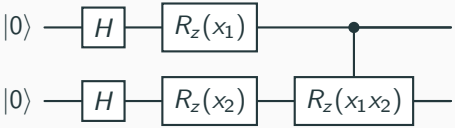
Product encoding versus entangling encoding

Product encoding:



Similarity factorizes strongly across qubits.

Entangling encoding:



Interaction terms enter the state geometry directly.

`lecture_05_demo.ipynb`, Demo 5: non-entangling versus entangling similarities.

ZZ-style interaction intuition

The Pauli matrix

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \qquad Z \otimes Z = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

A ZZ-style phase block has the form

$$e^{-i\varphi(x_i, x_j)\, Z \otimes Z}.$$

This is a quantum analogue of introducing pairwise interaction terms, but it does so through phases and interference rather than by appending one extra classical coordinate.

Classical parallel

Classical lecture-4 story:

$$(x_1, x_2) \longmapsto (x_1, x_2, x_1 x_2).$$

Quantum lecture-5 story:

$$(x_1, x_2) \longmapsto U_\phi(x_1, x_2) \left|00\right\rangle ,$$

where an entangling block makes the overlap depend on joint feature effects.

In both cases we are enriching geometry without manually enumerating every useful nonlinear feature in the original input space.

Pause D: solve Card Set D

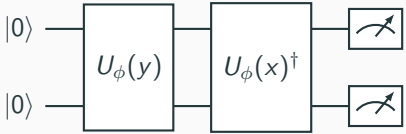
Estimating the kernel

Adjoint trick

For a feature map $|\phi(x)\rangle = U_\phi(x)\,|0\rangle^{\otimes n}$,

$$K_Q(x,y) = |\langle\phi(x)|\phi(y)\rangle|^2 = \left|\langle 0|\, U_\phi(x)^\dagger\, U_\phi(y)\,|0\rangle\right|^2.$$

Circuit picture:



The probability of measuring $0\ldots0$ is exactly the kernel value.

`lecture_05_demo.ipynb`, Demo 4: adjoint trick as a projection probability.

Why does $P(0\ldots0)$ equal the kernel?

Let

$$|\chi\rangle = U_\phi(x)^\dagger U_\phi(y)\,|0\rangle^{\otimes n}\,.$$

The Born rule gives

$$P(0\ldots0) = |\langle 0|\,\chi|^2 = \left|\langle 0|\,U_\phi(x)^\dagger U_\phi(y)\,|0\rangle\right|^2 = K_Q(x,y).$$

So the kernel is a projection probability onto the all-zero reference state.

SWAP test

Another route uses an ancilla qubit:

$$P(\mathrm{anc} = 0) = \frac{1+F}{2}, \qquad F = |\langle\phi(x)|\phi(y)\rangle|^2.$$

Operational picture:

1. prepare an ancilla in $|0\rangle$ and the two states $|\phi(x)\rangle$, $|\phi(y)\rangle$
2. apply Hadamard - controlled-SWAP - Hadamard on the ancilla
3. estimate $P(\mathrm{anc} = 0)$ and recover the fidelity

This is useful when we think operationally about fidelity estimation rather than directly composing $U_\phi(x)^\dagger U_\phi(y)$.

`lecture_05_demo.ipynb`, Demo 2 and Demo 4: fidelity and estimation routes.

Practical notes

Building a kernel matrix for N samples needs

$$\frac{N(N+1)}{2}$$

pairwise evaluations for a symmetric Gram matrix.

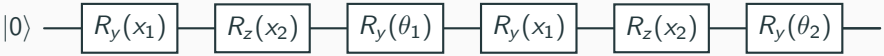
Practical caveats:

- shot noise perturbs the estimated similarities
- hardware noise changes overlaps and can wash out distinctions
- simulator studies are easier, but they do not remove the scaling cost

Pause E: solve Card Set E

Data re-uploading

Data re-uploading means encoding the same features more than once:



The idea is simple: data, trainable transformation, data again.

Repeated injection can create a richer effective feature map than a single-pass encoding.

`lecture_05_demo.ipynb`, Demo 7: one-qubit re-uploading preview.

Implementation workflow

Practical workflow:

1. choose an encoding circuit $U_\phi(x)$
2. compute pairwise similarities $K_{ij} = K_Q(x_i, x_j)$
3. compare against classical kernels under the same split
4. only then ask whether the quantum geometry helps

This lecture prepares the later quantum-kernel classifier material.

Transition to later lectures

Lecture 4 asked what a kernel is.

Lecture 5 asked how classical data becomes quantum states and how overlaps become kernels.

Later we will use these Gram matrices inside actual classifiers and compare them against strong classical baselines.